Аномалии в реляционных БД. Нормализация. Табличные выражения.

ст. преподаватель кафедры АСУ
Валова Анастасия Александровна

Аномалии в реляционных БД

Аномалии удаления

- Аномалии удаления кортежей
- Аномалии удаления атрибутов
- Аномалии удаления отношений

Employee_ID	Name	Department	Student_group	Discipline_name
123	Иванов И.И.	АСУ	БИВТ-26-1	Что-то про нейронки
234	Петров П.П.	АПД	БИВТ-26-2	Что-то про автокад
234	Петров П.П.	АПД	БИВТ-26-3	Что-то про фигму
456	Сидоров С.С.	БИСУП	БИВТ-26-4	Что-то про аналитику
456	Сидоров С.С.	БИСУП	БИВТ-26-1	Что-то про 1С

Group_ID	Student_name
БИВТ-26-1	…
БИВТ-26-1	…
БИВТ-26-2	…
БИВТ-26-3	…
…	…

Аномалии в реляционных БД

Аномалии вставки

- Аномалии вставки кортежей
- Аномалии вставки атрибутов
- Аномалии вставки отношений

Employee_ID	Name	Department	Student_group	Discipline_ID
123	Иванов И.И.	АСУ	БИВТ-26-1	Что-то про нейронки
234	Петров П.П.	АПД	БИВТ-26-2	Что-то про фигму
234	Петров П.П.	NULL	БИВТ-26-3	Что-то про фигму
456	Сидоров С.С.	БИСУП	БИВТ-26-4	Что-то про аналитику
456	Сидоров С.С.	БИСУП	БИВТ-26-1	-

Depart_ID	Department
1	АСУ
2	АПД
3	ИКТ
4	БИСУП
…	…

Аномалии в реляционных БД

Аномалии обновления (модификации)

- Аномалии вставки кортежей
- Аномалии вставки атрибутов
- Аномалии вставки отношений

Employee_ID	Name	Department	Student_group	Discipline_name
123	Новиков С.С.	АСУ	БИВТ-26-1	Что-то про нейронки
234	Петров П.П.	АПД	БИВТ-26-2	Что-то про автокад
234	Петров П.П.	АПД	БИВТ-26-3	Что-то про автокад
456	Сидоров С.С.	СУПиБИ	БИВТ-26-4	Что-то про аналитику
456	Сидоров С.С.	БИСУП	БИВТ-26-1	Что-то про 1С

Employee_ID	Discipline_name
234	Что-то про фигму
…	…

Денормализованное отношение

ФИО студент	Сем	Дисциплина	Форма контроля	Оценка	Ак.Ч	ФИО преподавателя
Петрова А.П.	1	Английский язык	зачет	2	60	Цветкова А.Ю.
Петрова А.П.	1	Мат. анализ	зачет	2	28	Рыбин К.К.
Петрова А.П.	1	Мат. анализ	экзамен	3	32	Раков И.И.
Петрова А.П.	1	Программирование	зачет	2	36	Незабудкина З.П.
Петрова А.П.	1	Программирование	экзамен	4	32	Зайчиков А.А.
Петрова А.П.	1	Линейная алгебра	зачет	2	24	Волков Г.И.
Петрова А.П.	1	Линейная алгебра	экзамен	4	28	Волков Г.И.
Петрова А.П.	1	История Отечества	экзамен	5	24	Москвин А.П.
Сидоров К.К.	3	Английский язык	зачет	2	60	Цветкова А.Ю.
Сидоров К.К.	3	Мат. анализ	зачет	2	20	Карпов К.Ю.
Сидоров К.К.	3	Мат. анализ	экзамен	5	28	Раков И.И.
Сидоров К.К.	3	Теория вероятностей и мат. статистика	экзамен	4	32	Соболев И.Г.
Сидоров К.К.	3	Операционные системы	зачет	2	36	Незабудкина З.П.
Сидоров К.К.	3	Операционные системы	экзамен	4	32	Незабудкина З.П.
Сидоров К.К.	3	Экономическая теория	зачет	2	24	Лабиринтов Е.Н.

Денормализованное отношение

ФИО студента	Сем	Дисциплина	Форма контроля	Оценка	Ак.ч.	ФИО преподавателя
Петрова А.П.	1	Английский язык	зачет	2	60	Цветкова А.Ю.
Петрова А.П.	1	Мат. анализ	зачет	2	28	Рыбин К.К.
Петрова А.П.	1	Мат. анализ	экзамен	3	32	Раков И.И.
Петрова А.П.	1	Программирование	зачет	2	36	Незабудкина З.П.
Петрова А.П.	1	Программирование	экзамен	4	32	Зайчиков А.А.
Петрова А.П.	1	Линейная алгебра	зачет	2	24	Волков Г.И.
Петрова А.П.	1	Линейная алгебра	экзамен	4	28	Волков Г.И.
Петрова А.П.	1	История Отечества	экзамен	5	24	Москвин А.П.
Сидоров К.К.	3	Английский язык	зачет	2	60	Цветкова А.Ю.
Сидоров К.К.	3	Мат. анализ	зачет	2	20	Карпов К.Ю.
Сидоров К.К.	3	Мат. анализ	экзамен	5	28	Раков И.И.
Сидоров К.К.	3	Теория вероятностей и мат. статистика	экзамен	4	32	Соболев И.Г.
Сидоров К.К.	3	Операционные системы	зачет	2	36	Незабудкина З.П.
Сидоров К.К.	3	Операционные системы	экзамен	4	32	Незабудкина З.П.
Сидоров К.К.	3	Экономическая теория	зачет	2	24	Лабиринтов Е.Н.

Проблемы с несогласованной базой | | (Аномалии вставки, удаления и обновления)

1. Все аномалии демонстрируют, что:

- Объединение информации о разных объектах предметной области в одну таблицу может вызвать проблемы.

2. Чтобы смягчить эти аномалии, важно:

- Разработать хорошо структурированную схему базы данных.
- Определить соответствующие взаимосвязи и ограничения.
- Обеспечить нормализацию для устранения избыточности и поддержания целостности данных.

Нормализация. Декомпозиция

Нормализация отношения - это декомпозиция исходного отношения на набор других отношений, обладающих меньшей степенью.

Декомпозиция позволяет эффективнее выполнять операции модификации данных, т.к. сокращается число проверок и вспомогательных действий, поддерживающих целостность БД.

Процесс декомпозиции – это замена отношения некоторым набором его проекций.

Декомпозиция должна быть обратимой, т. е. должна иметься возможность “собрать” исходное отношение из декомпозированных отношений без потери информации (декомпозиция без потерь).

Нормальные формы. 1НФ

Отношение находится ***в первой нормальной форме (1НФ)*** тогда и только тогда, когда это отношение содержит только **простые** и **однозначные атрибуты**, причем обязательно с одной и той же *семантикой*.

- Не допускается дублирование строк в таблице.

Заказ	Клиент	Информация о заказе
1	Alice	2x Laptop, 3x Mouse
2	Bob	1x Smartphone, 1x Charger



Заказ	Клиент	Продукт	Количество
1	Alice	Laptop	2
1	Alice	Mouse	3
…	…	…	…

Составные атрибуты – это атрибуты, составленные из нескольких простых атрибутов.
Многозначные атрибуты – это атрибуты, представляющие множество значений.

*Фирма	Модели
BMW	M5, X5M, M1
Nissan	GT-R



*Фирма	Модели
BMW	M5
BMW	X5M
BMW	M1
Nissan	GT-R

Функциональная зависимость

Функциональная зависимость описывает связь между атрибутами отношения: если в отношении R, содержащем атрибуты **A** и **B**, атрибут **B** функционально зависит от атрибута **A**, то каждое отдельное значение атрибута **A** связано только с одним значением атрибута **B** (причем в качестве **A** и **B** могут выступать группы атрибутов).
A - ***детерминант*** функциональной зависимости

P.S. Атрибут, который однозначно определяет значение другого атрибута

A→B ➡ **Сотрудник**→Должность ➡ **«Иванов И.И.»**→ «Профессор»

EmployeeID	Name	Position
1	Alice	Manager
2	Bob	Developer
3	Charlie	Developer
4	David	Manager

EmployeeID→Name,Position

Position↛EmployeeID

Функциональные зависимости,.

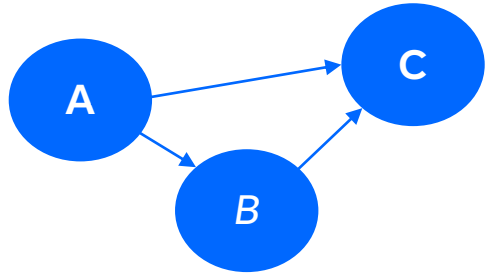
Транзитивная зависимость

A→B является ***полной***, если удаление какого-либо атрибута из группы атрибутов A приводит к потере этой зависимости

A→B является ***частичной***, если в группе атрибутов A есть один или несколько атрибутов, при удалении которых эта зависимость сохраняется

Транзитивная зависимость описывает связь между атрибутами отношения: если в отношении R, содержащем атрибуты A, B и C, атрибут B функционально зависит от атрибута A, а атрибут C функционально зависит от атрибута B, то атрибут C транзитивно зависит от атрибута A.

A→B, B→C - атрибут **C** связан ***транзитивной зависимостью*** с атрибутом **A** через атрибут B



Нормальные формы. 2НФ

Отношение находится ***во второй нормальной форме (2НФ),*** если оно удовлетворяет определению 1НФ и каждый неключевой атрибут функционально полно зависит от *каждой части* первичного ключа.

Неключевой атрибут – это любой атрибут отношения, не содержащийся в каком-либо первичном ключе или ключе-кандидате отношения.

Полная функциональная зависимость от ключа – это отсутствие функциональной зависимости от какой-либо части этого ключа.

Модель*	Фирма*	Цена	Скидка
M5	BMW	5500000	5%
X5M	BMW	6000000	5%
M1	BMW	2500000	5%
GT-R	Nissan	5000000	10%



Модель*	Фирма*	Цена
M5	BMW	5500000
X5M	BMW	6000000
M1	BMW	2500000
GT-R	Nissan	5000000

Фирма*	Скидка
BMW	5%
Nissan	10%

Нормальные формы. 3НФ

Отношение находится ***в третьей нормальной форме (3НФ),*** если оно удовлетворяет определению 2НФ и каждый неключевой атрибут функционально полно зависит только от ключей (первичного и ключей кандидатов).

== каждый неключевой атрибут нетранзитивно функционально зависит от первичного ключа.

Не допускается наличие транзитивных функциональных зависимостей вида **A**→B, B→**C**.

Модель*	Автосалон	Телефон
BMW	Независимость	12-34-56
Audi	Независимость	12-34-56
Nissan	Genser	65-43-21



Модель*	Автосалон
BMW	Независимость
Audi	Независимость
Nissan	Genser

Автосалон*	Телефон
Независимость	12-34-56
Genser	65-43-21

Нормальные формы. НФ Бойса-Кодда

Отношение находится ***в нормальной форме Бойса-Кодда (НФБК, или усиленная ЗНФ)*** тогда и только тогда, когда она находится в ЗНФ и любая функциональная зависимость между ее атрибутами сводится к полной функциональной зависимости от *возможного* первичного ключа.

Отличие от ЗНФ: не только любой неключевой атрибут полностью функционально зависит от любого ключа, но и любой ключевой атрибут должен полностью функционально зависеть от любого ключа.

Стоянка	Начало	Окончание	Тариф
1	9:30	10:30	Бережливый
1	11:00	12:00	Бережливый
1	14:00	15:30	Стандарт
2	10:00	12:00	Премиум В
2	12:00	14:00	Премиум В
2	15:00	18:00	Премиум А

Ключи: {Стоянка, Начало}, {Стоянка, Окончание}, {Тариф, Начало}, {Тариф, Окончание}.

Тариф*	Стоянка
Бережливый	1
Стандарт	1
Премиум А	2
Премиум В	2

Тариф*	Начало*	Окончание
Бережливый	9:30	10:30
Бережливый	11:00	12:00
Стандарт	14:00	15:30
Премиум В	10:00	12:00
Премиум В	12:00	14:00
Премиум А	15:00	18:00

Нормальные формы. НФ Бойса-Кодда

№ Курса	Преподаватель	Аудитория
101	Иванов И·И·	801
101	Сидоров С·С·	801
102	Петров П·П·	187
102	Иванов И·И·	781



№ Курса*	Аудитория
101	801
102	187
102	781

Преподаватель*	№ Курса*
Иванов И·И·	101
Сидоров С·С·	101
Петров П·П·	102
Иванов И·И·	102

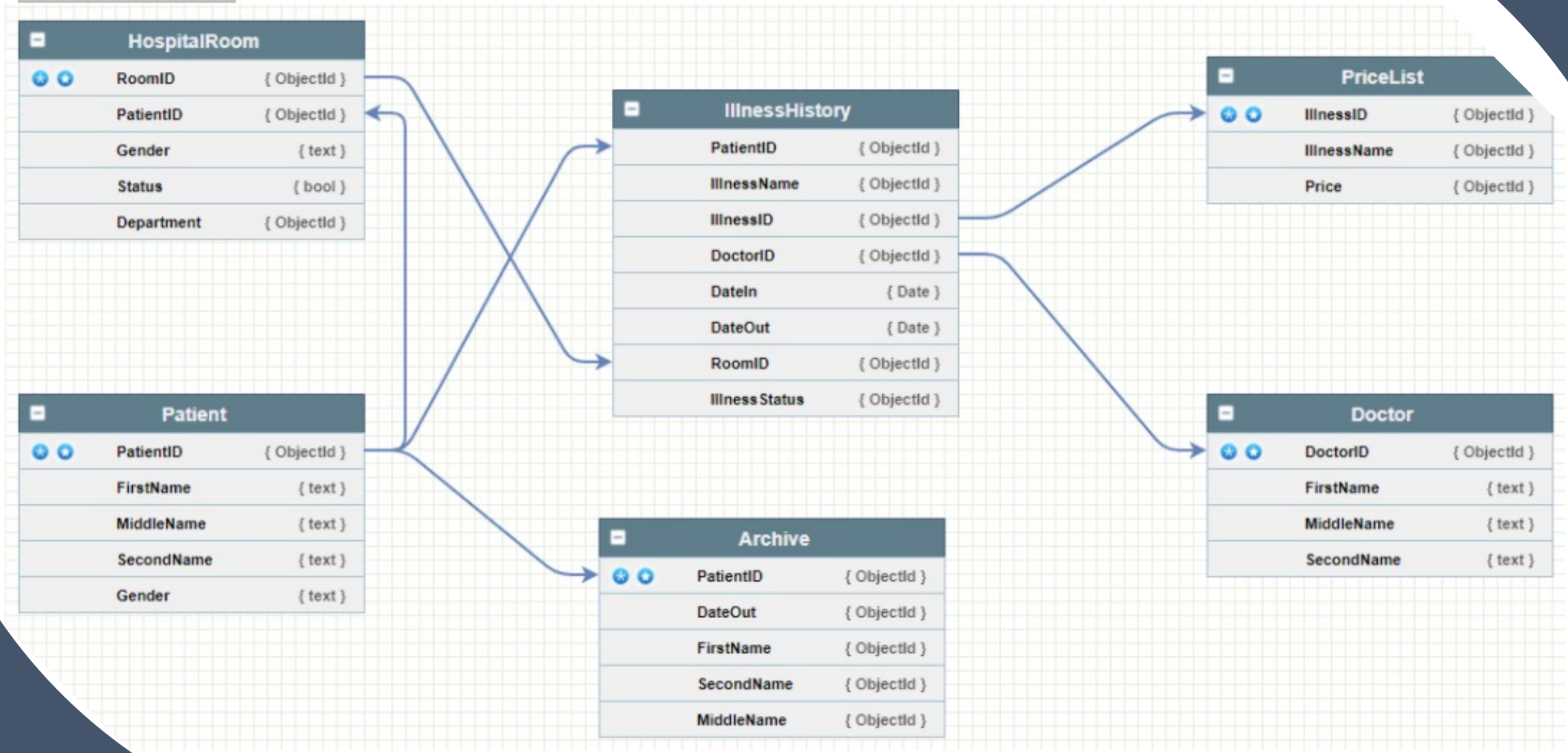
Функциональные зависимости

A→B ➡ **Курс**→Аудитория

A→B ➡ **Преподаватель, Аудитория**→Курс

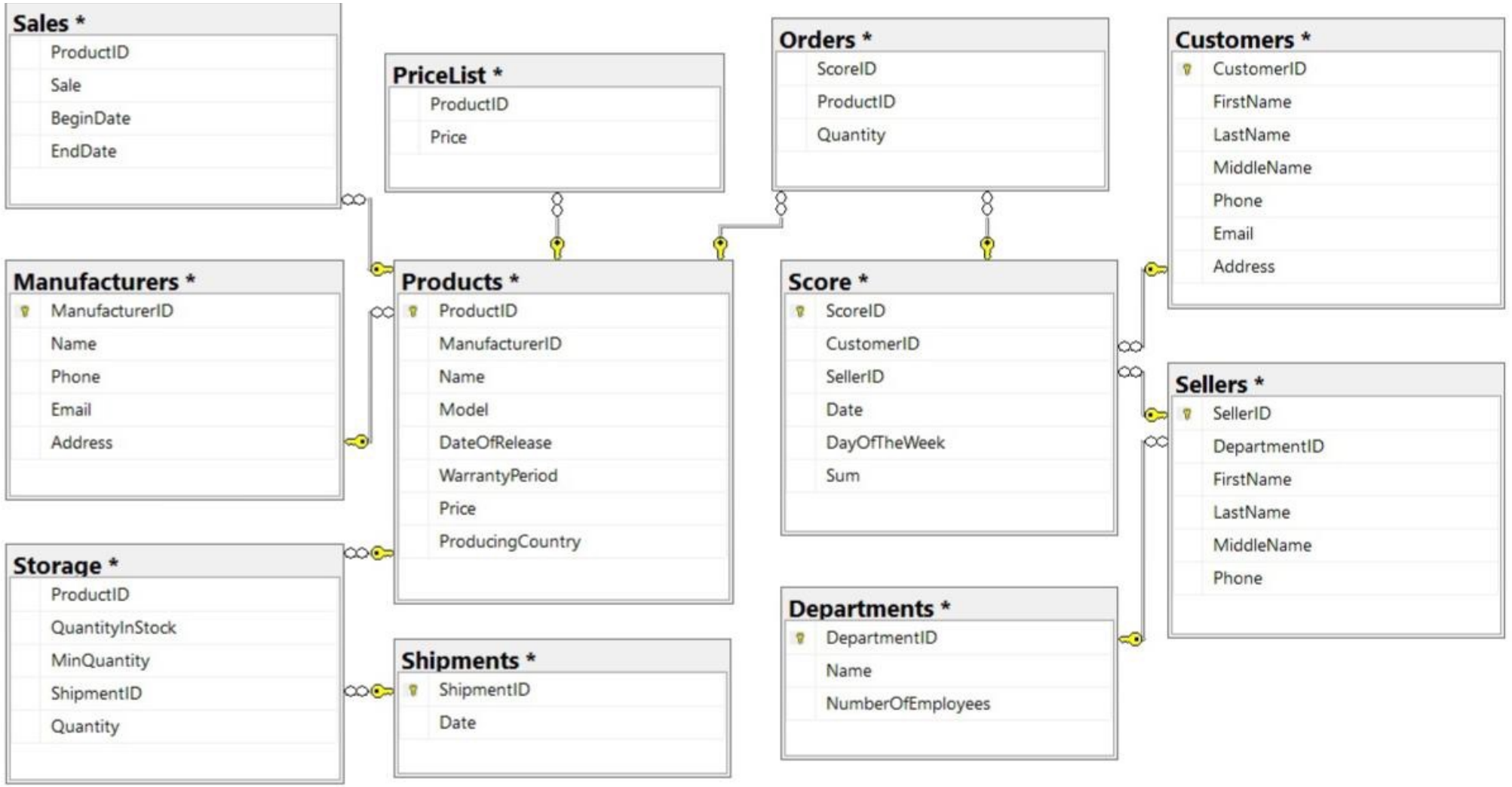
Примеры ненормализованных БД

так не делать



Примеры ненормализованных БД

так не делать



7 Использование табличных выражений

Что такое представление (View)?

- Объект БД, на который можно ссылаться, как на таблицу.
  - CREATE VIEW название_представления AS выражение_SELECT
- Именованный SELECT-запрос:

```
CREATE VIEW vEmployeeDepartments AS
    SELECT e.Name, d.DepartmentName
    FROM Employees e JOIN Departments d
    ON e.DepartmentID = d.DepartmentID;
```

Введение в представления

- Представление не сохраняет данные (кроме индексированных).
- Добавление уникального кластерного индекса делает представление индексированным:
  - Данные сохраняются на диск, увеличивая производительность
- **WITH SCHEMABINDING** блокирует изменения в схеме базовых таблиц.
- Вставка и обновление данных в представление может происходить только в одной базовой таблице.

Запросы к представлениям

- Представления (отображения) – именованные запросы с хранимым в БД определением:
  - Обеспечивают абстракцию, инкапсуляцию и упрощение;
  - С точки зрения администратора, отображения обеспечивают дополнительный уровень безопасности.
- Используются в запросах SELECT как таблицы:

```
CREATE VIEW Sales.vSalesOrders
AS
SELECT  oh.OrderID, oh.Orderdate, oh.CustomerID,
        od.ProductID, od.Quantity
FROM Sales.OrderHeaders AS oh
INNER JOIN Sales.OrderDetails AS od
ON od.OrderID = oh.OrderID;
```

```
SELECT OrderID, CustomerID, ProductID
FROM Sales.vSalesOrder
ORDER BY OrderID;
```

Временные таблицы

- Временные таблицы предназначены для хранения временных результатов **в рамках сессии** пользователя:
  - Создаются в БД tempdb и удаляются автоматически;
  - Создаются с префиксом “**#**”;
  - Глобальные временные таблицы создаются с префиксом “**##**”.

```
CREATE TABLE #tmpProducts
(ProductID INTEGER,
 ProductName varchar(50));
GO
…
SELECT * FROM #tmpProducts;
```

Табличные переменные

- Введены из-за частых рекомпиляций временных таблиц;
- Используются как временные таблицы, но видимость ограничена **текущим пакетом**;
- Используются только на малых наборах данных.

```
DECLARE @varProducts table
(ProductID INTEGER,
 ProductName varchar(50));
…
SELECT * FROM @varProducts
```

Табличные функции

- TF – именованные объекты с определениями хранимыми в БД;
- TF возвращают виртуальную таблицу в точку вызова;
- В отличие от представлений, TF поддерживают входные параметры:
  - TF можно определить, как параметризованные представления.

```
CREATE FUNCTION Sales.fn_GetOrderItems (@OrderID AS Integer)
RETURNS TABLE
AS
RETURN
(SELECT ProductID, UnitPrice, Quantity
FROM Sales.OrderDetails
WHERE OrderID = @OrderID);
…
SELECT * FROM Sales.fn_GetOrderItems (1025) AS LineItems;
```

Основы пользовательской табличной функции

- Предложение RETURNS также определяет формат таблицы. Область видимости табличной переменной локализована внутри функции.
- Операторы INSERT Transact-SQL в теле функции формируют ответ, добавляя строки в возвращаемую переменную, определенную в предложении RETURN.
- При выполнении оператора RETURN данные из табличной переменной возвращаются в виде результирующей таблицы. Оператор RETURN не может иметь аргументов.

```
CREATE FUNCTION GetEmployees(@DeptID INT)
RETURNS @Result TABLE
            ( EmployeeID INT,
            Name NVARCHAR(50),
            Salary MONEY -- ← RETURNS определяет структуру таблицы )


AS BEGIN
            -- @Result доступна ТОЛЬКО здесь - область видимости локализована
            RETURN
 END
```

Производные таблицы (Derived Tables)

Введение

- Производные таблицы – именованные запросы, созданные в рамках внешнего оператора SELECT.
- Не хранятся в БД – представляют виртуальную таблицу.
- Область видимости производной таблицы – запрос, в котором она определена.

```
SELECT orderyear, COUNT(DISTINCT custid) AS cust_count
FROM
        (SELECT YEAR(orderdate) AS orderyear, custid
        FROM Sales.Orders) AS derived_year
GROUP BY orderyear;
```

Производные таблицы.
Основные правила

- Производные таблицы **обязаны**:
  - Иметь псевдоним;
  - Иметь уникальные имена для всех столбцов;
  - Не использовать предложение ORDER BY clause (без TOP или OFFSET/FETCH);
  - Избегать множественных ссылок в том же запросе.
- Производные таблицы **могут**:
  - Использовать внутренние или внешние псевдонимы для столбцов;
  - Ссылаться на параметры и/или переменные;
  - Быть вложенными в другие производные таблицы.

Производные таблицы

Указание псевдонимов столбцов: 2 варианта

- Псевдонимы столбцов могут объявляться внутри:

```
SELECT orderyear, COUNT(DISTINCT custid) AS cust_count
FROM (  SELECT YEAR(orderdate) AS orderyear, custid
        FROM Sales.Orders) AS derived_year
GROUP BY orderyear;
```

- Или снаружи:

```
SELECT orderyear, COUNT(DISTINCT custid) AS cust_count
FROM (  SELECT YEAR(orderdate), custid
        FROM Sales.Orders) AS
        derived_year(orderyear, custid)
GROUP BY orderyear;
```

Обобщенные табличные выражения (CTE)

- CTE – именованные табличные выражения, определенные в запросе;
- CTE похожи на производные таблицы по области видимости и требованиям к именованию;
- В отличие от производных таблиц, CTE поддерживают множественные ссылки и рекурсию.

```
WITH CTE_year (OrderYear, CustID)
AS
(
        SELECT YEAR(orderdate), custid
        FROM Sales.Orders
)
SELECT OrderYear, COUNT(DISTINCT CustID) AS Cust_Count
FROM CTE_year
GROUP BY orderyear;
```

Обобщенные табличные выражения (CTE)

Рекурсия

- Укажите корневой запрос;
- Используйте UNION ALL, чтобы добавить рекурсивный запрос;
- Запросы к CTE могут использовать опцию MAXRECURSION.

```
WITH OrgReport (ManagerID, EmployeeID, EmployeeName, Level)
AS
(
        SELECT e.ManagerID, e.EmployeeID, EmployeeName, 0
        FROM HR.Employee AS e
        WHERE ManagerID IS NULL
        UNION ALL
        SELECT e.ManagerID, e.EmployeeID, e.EmployeeName, Level + 1
        FROM HR.Employee AS e
        INNER JOIN OrgReport AS o ON e.ManagerID = o.EmployeeID
)
SELECT * FROM OrgReport
OPTION (MAXRECURSION 3);
```

8 Множества группировок и поворот данных

Множества группировок

Синтаксис

- Раздел GROUPING SETS основан на предложении GROUP BY;
- Позволяет задавать множество группировок в одном запросе.

```
SELECT <список столбцов с агрегатами>
FROM <источник>
GROUP BY
GROUPING SETS
(
        <column_name>, -- один или более столбцов
        <column_name>, -- один или более столбцов
        () -- круглые скобки, если надо агрегировать все строки
);
```

Множества группировок

Пример

	EmployeeID	CustomerID	TotalAmount
Итого для всех продаж	*NULL*	*NULL*	256.23
Подытог для каждого клиента	*NULL*	1	49.99
	NULL	2	107.49
	NULL	3	98.75
Подытог для каждого сотрудника	1	*NULL*	107.49
	2	*NULL*	148.74

```
SELECT EmployeeID, CustomerID, SUM(Amount) AS TotalAmount
FROM Sales.SalesOrder
GROUP BY
GROUPING SETS(EmployeeID, CustomerID,());
```

ROLLUP и CUBE

- ROLLUP обеспечивает множество группировок с комбинациями в иерархическом стиле:

```
SELECT StateProvince, City, COUNT(CustomerID) AS Customers
FROM Sales.vCustomerDetails
GROUP BY ROLLUP(StateProvince, City)
ORDER BY StateProvince, City;
```

- CUBE обеспечивает множество группировок с перебором всех возможных комбинаций для группировки:

```
SELECT SalesPersonName, CustomerName, SUM(Amount) AS TotalAmount
FROM Sales.vSalesOrders
GROUP BY CUBE(SalesPersonName, CustomerName)
ORDER BY SalesPersonName, CustomerName;
```

Идентификация группировок в результатах

- Несколько группировок затрудняют идентификацию источника каждой строки в результирующем наборе;
- NULL может появиться из данных или представлять значение-заполнитель в группировке;
- Через функцию GROUPING_ID можно определить группировку по столбцу (результат 1 или 0):

```
SELECT GROUPING_ID(SalesPersonName) AS SalesPersonGroup,
       GROUPING_ID(CustomerName) AS CustomerGroup,
       SalesPersonName, CustomerName, SUM(Amount) AS TotalAmount
FROM Sales.vSalesOrders
GROUP BY CUBE(SalesPersonName, CustomerName)
ORDER BY SalesPersonName, CustomerName;
```

GROUPING_ID для нескольких столбцов

SELECT a, b, c, SUM(d), GROUPING_ID(a,b,c)
FROM T GROUP BY <список group by>

Статистически обработанные столбцы	**Входные данные GROUPING_ID (a, b, c) = GROUPING(a) + GROUPING(b) + GROUPING(c)**	**Выходные данные GROUPING_ID ()**
a	100	4
b	010	2
c	001	1
ab	110	6
ac	101	5
bc	011	3
abc	111	7

GROUPING_ID () возвращает битовое представление типа integer, в котором N младших битов могут быть установлены (1).

Установленный бит означает, что соответствующий аргумент не является столбцом группирования для указанной выходной строки.

Применительно к одиночному запросу группировки функция GROUPING (<выражение-столбец>) эквивалентна GROUPING_ID (<выражение-столбец>), и обе эти функции возвращают 0.

Поворот данных (PIVOT)

- Поворот данных – это транспонирование данных из построчной ориентации в ориентацию на основе столбцов;
- Уникальные значения из одного столбца проецируются в заголовки столбцов — может происходить агрегация:

OrderID	ProductID	Category	Revenue
1023	1	Bikes	1078.75
1023	15	Accessories	52.00
1023	21	Accessories	124.90
1024	1	Bikes	2491.00
1025	3	Bikes	1067.49
1025	15	Accessories	125.99
1025	35	Clothing	26.57
1025	36	Clothing	5.78

OrderID	Bikes	Accessories	Clothing
1023	1078.75	176.90	*NULL*
1024	2491.00	*NULL*	*NULL*
1025	1067.49	125.99	32.35

```
SELECT OrderID, Bikes, Accessories, Clothing
FROM
    (SELECT OrderID, Category, Revenue FROM Sales.SalesDetails) AS sales
PIVOT (SUM(Revenue) FOR Category IN([Bikes], [Accessories], [Clothing]))
AS pvt
```

Обратный поворот данных (UNPIVOT)

- Обратный поворот – это поворот данных из ориентации на основе столбцов в ориентацию на основе строк;
- Распространяет или разделяет значения из одной строки-источника в одну или несколько целевых строк;
- Каждая строка-источник становится одной или несколькими строками в результате, основанном на количестве поворотных столбцов:

OrderID	Bikes	Accessories	Clothing
1023	1078.75	176.90	*NULL*
1024	2491.00	*NULL*	*NULL*
1025	1067.49	125.99	32.35

```
SELECT OrderID, Category, Revenue
FROM
      (SELECT OrderID, Bikes, Accessories, Clothing FROM Sales.SalesByCat) AS pvt
UNPIVOT (Revenue FOR Category IN([Bikes], [Accessories], [Clothing])) AS unpvt
```

OrderID	Category	Revenue
1023	Bikes	1078.75
1023	Accessories	176.90
1024	Bikes	2491.00
1025	Bikes	1067.49
1025	Accessories	125.99
1025	Clothing	32.35